

Tema 6:
TRIES

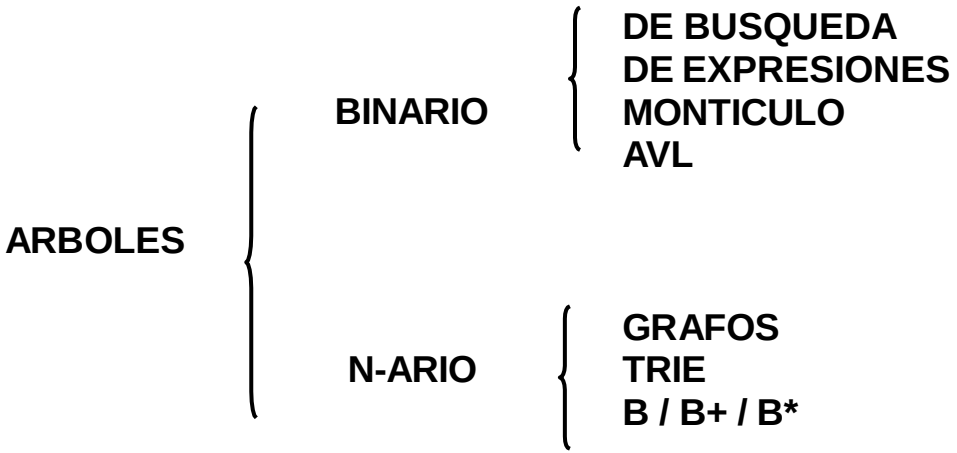
Estructura de Datos

Carreras: Analista de Sistemas (2do. año)
Licenciatura en Sistemas

Universidad Nacional de la Patagonia Austral
Unidad Académica Río Gallegos

repaso

ARBOLES



TRIE
Arbol de recuperación
reTRIEval

- Introducidos en 1959 por Rene de la Briandais y Edward Fredkin

- Es un árbol N-ario

- Es una estructura que se utiliza para representar conjuntos de cadenas de caracteres, y recuperar información.

- El mismo método funciona para la representación de tipos de datos que son cadenas de objetos de cualquier tipo, como cadenas de enteros.

Aplicación

Los árboles TRIE son estructuras adecuadas cuando se necesitan manejar conjuntos grandes de cadenas que tienen la misma secuencia de letras, es decir, el mismo **prefijo**.

Ejemplos:

a) guía telefónica

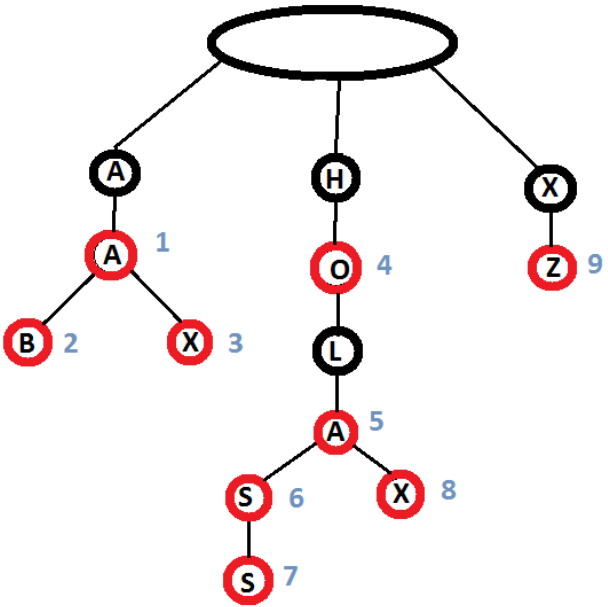
b) diccionario de términos

Aplicación

Cuando el número de prefijos distintos entre todas las cadenas del conjunto es mucho menor que la longitud total de todas las palabras, el TRIE es una estructura adecuada, en cuanto al tiempo de recuperación y espacio ocupado.

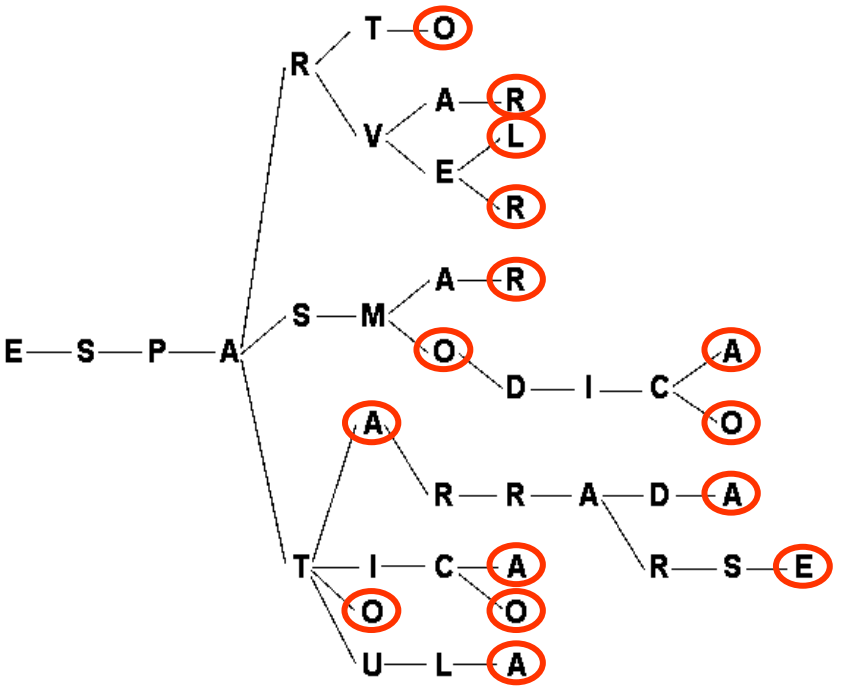
Ejemplo

Hola
Aab
Holas
Holass
Xz
Aa
aax
Holax
ho



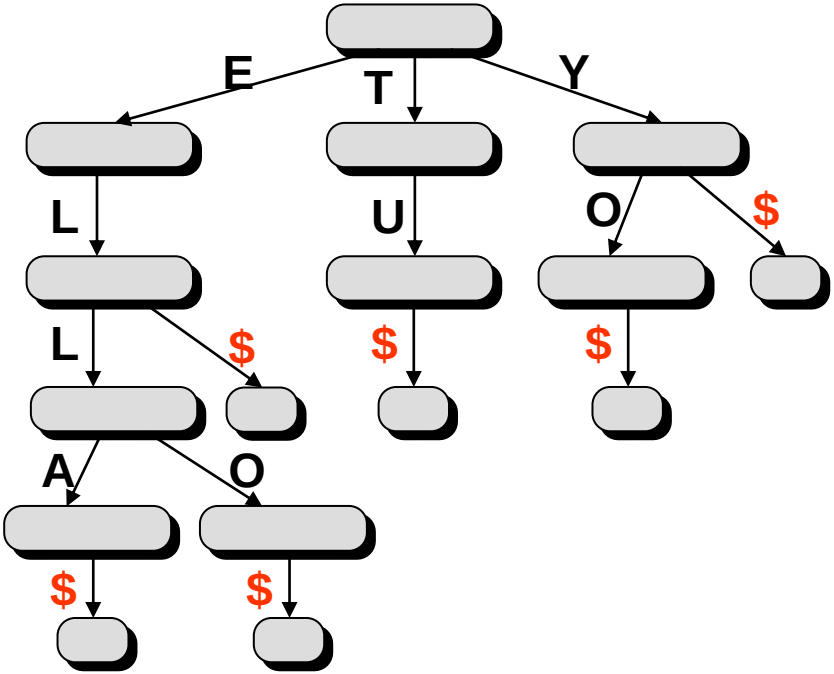
Árboles Trie: Introducción

Idea: muchas palabras tienen prefijos comunes. P. ej.: espasmar, espasmo, espasmódico, espasmódica, ...



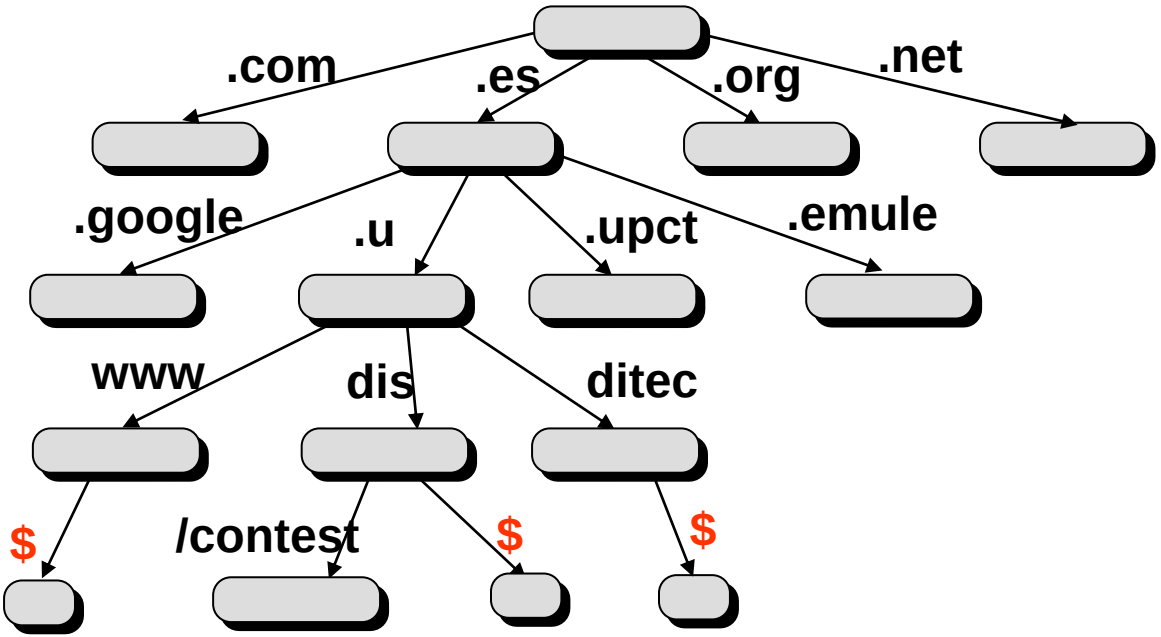
Árboles Trie: APLICACION

- Ejemplo, C= {ELLA, ELLO, EL, TU, Y, YO}

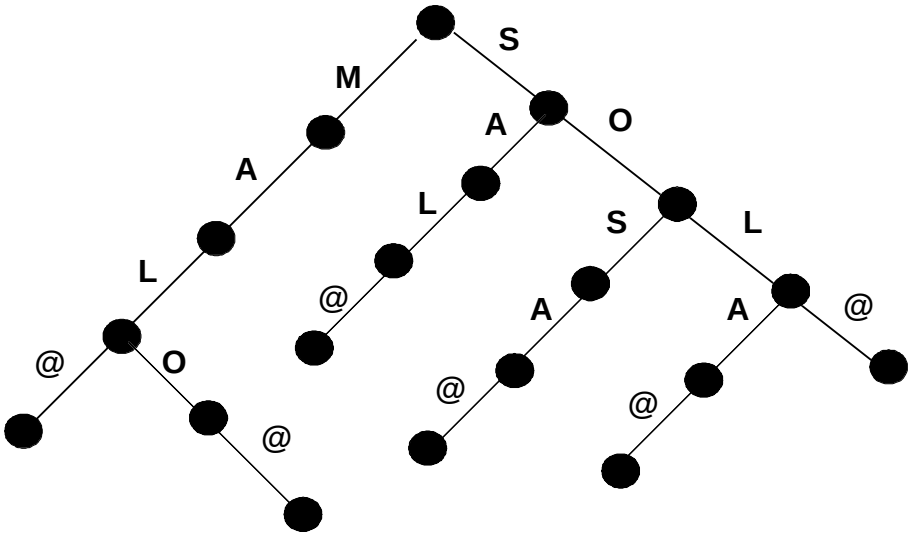


Árboles Trie: APLICACION

Se pueden representar otros tipos de información, cambiando el alfabeto **A**.
Ejemplo: representación de URL de páginas web.



TRIE GRAFICAMENTE



{ MAL, MALO, SOL, SOLA,SOSA,SAL }

Características

- En un TRIE cada camino de la raíz a una hoja corresponde a una palabra del conjunto representado.
- El final de cada palabra se utiliza un carácter (@) que indica el final de la palabra
- La raíz corresponde a la cadena vacía y sus dos hijos M y S (prefijos)

Tener en cuenta:

1) Cada nodo tiene hasta 27 hijos, uno para cada letra

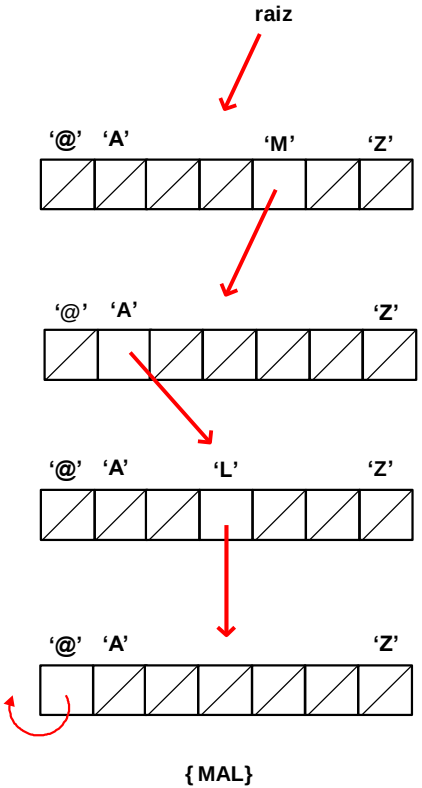
2) La mayor parte de los nodo tiene mucho menos de 28 hijos

3) Una hoja que sigue a una arista etiquetada con @ no puede tener hijos e incluso podría no existir

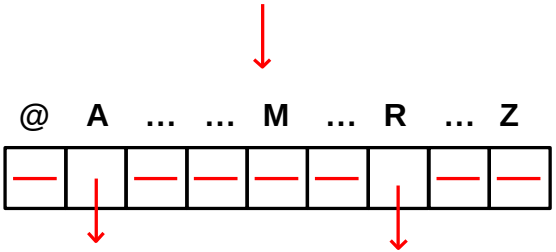
Implementación:

- Arreglos de punteros
- Listas enlazadas
- Matriz

Arreglos de punteros

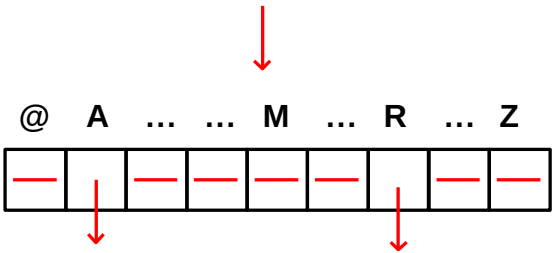


Características:

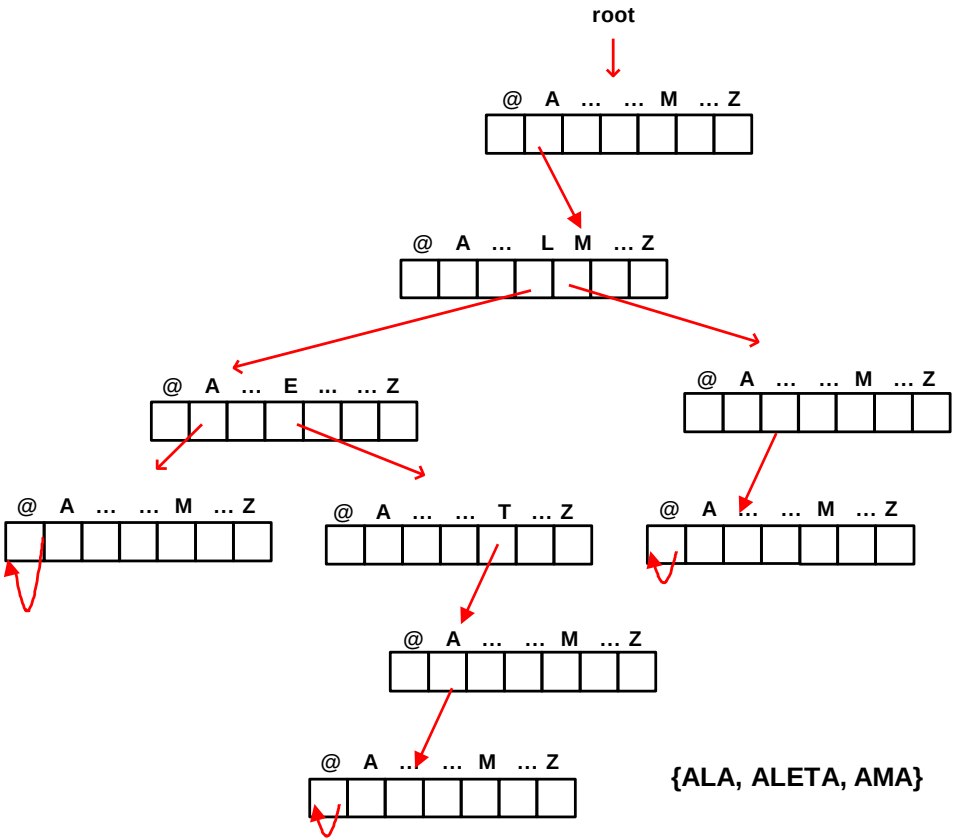


- Cada nodo es un arreglo de punteros
- Las posiciones de los nodos indican “caracteres”
- Una palabra de n caracteres, requiere n + 1 nodos
- La posición del arreglo estará nula si la palabra no contiene a ese carácter, dónde el nivel del nodo se corresponde con el orden del carácter en la palabra
- La posición del arreglo no estará nula si la palabra contiene a ese carácter, dónde el nivel del nodo se corresponde con el orden del carácter en la palabra

- El fin de la palabra se marca cuando el último nodo en la posición del carácter @ apunta así mismo.



- Es una combinación de implementación estática-dinámica.


root
@ A … … M … Z
@ A … L M … Z
@ A … E … … Z
@ A … … M … Z
@ A … … M … Z
@ A … … T … Z
@ A … … M … Z
@ A … … M … Z
@ A … … M … Z
{ALA, ALETA, AMA}

Diseño

- **TrieNode**: clase que representa cada nodo y su comportamiento
- **Trie**: clase que representa el arbol y su comportamiento (agregar palabras, buscar palabras, eliminar palabras, etc.)
- **Se debe incorporar un mecanismo para hacer** corresponder caracteres a posiciones
- Clase String: método length(), método charAt()

```
class TrieNode {
   TrieNode [] chars; // array de punteros a arrays

   TrieNode(int end)
   { chars = new TrieNode[end+1]; }

   TrieNode getValueAt(int pos)
    // devuelve el puntero a un nodo trie que esta en pos
   { return chars[pos]; }

   void setValueAt(int pos, TrieNode newNode)
     // modifica el puntero que esta en pos
   { chars[pos] = newNode; }
  }
```

```
class Trie {
   TrieNode root;
   final int end=26;
```

```
   final int begin=0;

Trie() // crea el arbol vacio
{ root = null; }


static int getPosition(char c){
   switch(c) {
     case '@': return 0;
     case 'a': return 1;
     case 'b': return 2;
     case 'c': return 3;
     ….
     ….
     ….
}
 void insertWord (String word){
   TrieNode t;
   int i, pos;
   if (root==null)                    // esta vacio
     { root = new TrieNode(end);
```

```
    t = root;
    i=0;
    while (i<word.length()) {
      pos = getPosition(word.charAt(i));
      if (t.getValue(pos)==null)
             t.setValueAt(pos, new TrieNode(end));
      t = t.getValueAt(pos);
      i++;
    }
    t.setValueAt(begin, t);
}

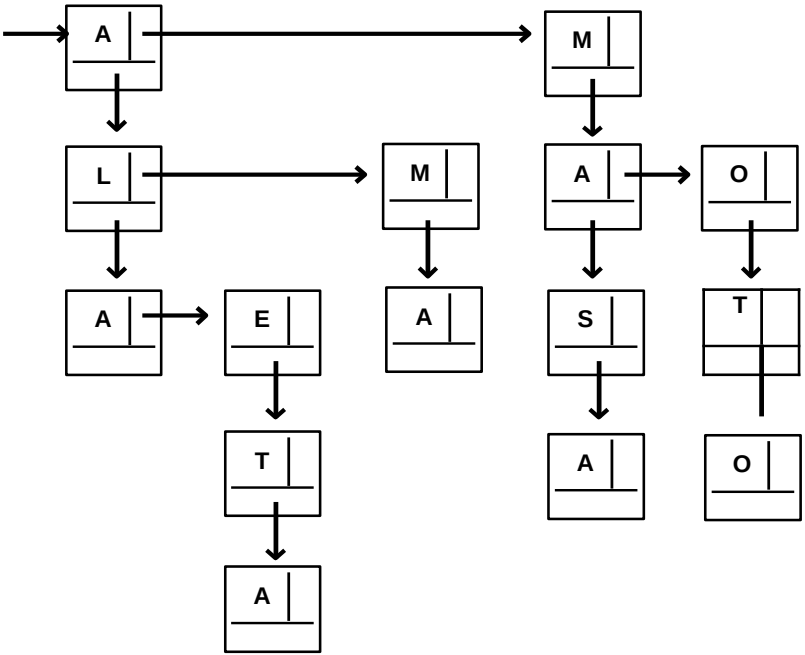


boolean searchWord(String word){
  TrieNode t=root;
  int i=0;
  if (t==null)
     return false;
  int pos;
  while (i<word.length()) {
```

```
      pos = getPosition(word.charAt(i));
      if (t.getValueAt(pos)!=null)
         { t=t.getValueAt(pos);
           i++; }
      else
         return false;
   }
   if (t.getValueAt(begin)==t)
      return true;
   else
      return false;
}
```

Listas Enlazadas

{ ALA, ALETA, AMA, MASA, MOTO }



CARACTERISTICAS

- Las LE están ordenadas
- Las palabras se almacenan por niveles
- Cada nodo tiene dos punteros: el derecho apunta al siguiente carácter de un prefijo diferente; el “sig” apunta al siguiente carácter del prefijo;
- Es totalmente dinámica
- Requiere menos espacio
- Requiere más tiempo
- Los algoritmos son más complejos

Representación con Matriz

Claves: hija, hijo, hoja, lata, latita, leo y liz.

	@	A	B	C	D	E	F	G	H	I	J	K	L	...	O	...	T	...	Z
0									1				12						
1										2					8				
2											3								
3		4													6				
4	5																		
5																			
6	7																		
7																			
8											9								
9		10																	
10	11																		
11																			
12		13				21				24									
13																	14		
14		15								17									
15	16																		
16																			
17																	18		
18		19																	
19	20																		
20																			
21															22				
22	23																		
23																			
24																			25
25	26																		
26																			

CARACTERISTICAS

- Cada renglón (fila) corresponde a un estado y cada columna a un símbolo de entrada, incluyendo al de fin de cadena (#).
- Si G es el arreglo bidimensional, entonces G[n][a] contiene el estado al que se llega partiendo del estado n recorriendo el arco a.
- La búsqueda es sumamente rápida, pero consume mucho espacio y tiene limites
- Es estática.

Tiempo de ejecución

- Cual es el principal factor en el tiempo de ejecución?
  - la longitud de las palabras: **N**.
  - Nodos con **arrays**: **O(N)**
  - Nodos con **listas**: **O(N*s)**, donde **s** es la longitud promedio de las listas. En la práctica, ~ **O(N)**.

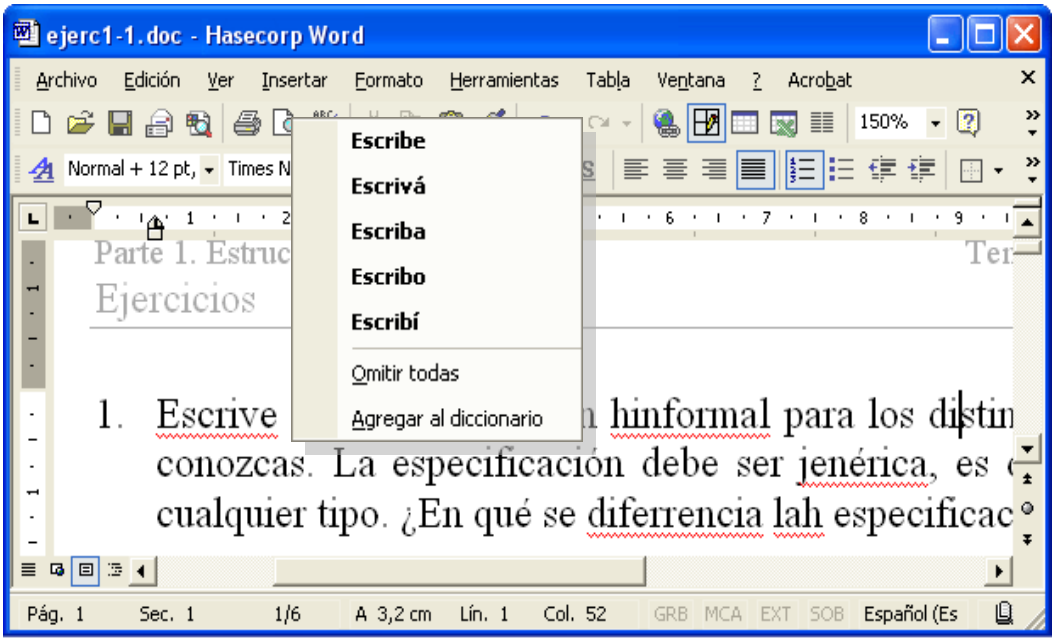
VENTAJAS

- Búsqueda de claves mas rápida que un ABB.
  - La búsqueda de una clave de longitud m tendrá en el peor de los casos un coste de O(n).
- menor espacio requerido para almacenar gran cantidad de cadenas pequeñas
  - las claves no se almacenan explícitamente
- Mayor eficiencia de algoritmos de búsquedas.

APLICACIONES

Aplicación: representación de diccionarios (o en general conjuntos) grandes de palabras.

Ejemplo. Corrector ortográfico interactivo.



APLICACIONES

Diccionario español: ~ 3 millones de palabras.

Muchas palabras → Mucha memoria y operaciones lentas.

Pero la búsqueda de una palabra no puede tardar más de 1 milisegundo...

... esparto esparvar esparvel esparver espasmar espasmo espasmódica espasmódico espata espatarrada espatarrarse espática espático espato espátula espatulomancia espaviento espavorecida espavorecido espavorida espavorido espay especería especia especial ...

Teléfonos móviles: Ingreso de texto predictivo.



Aplicaciones

A) Como sustitución de otras estructuras de datos

Como sustitución de una Tabla hash

Un Trie puede usarse para reemplazar una Tabla hash, sobre la que presenta las siguientes ventajas:

- en un trie no se producen colisiones de claves
- no hay que definir una función de hash, o modificarla si añadimos más claves
- los contenedores que almacenan distintos valores asociados a una única clave sólo son necesarios si tenemos más de un valor asociada a una única clave. En una tabla hash siempre se necesitan estos contenedores para las colisiones de clave

- un trie puede proporcionar un ordenamiento alfabético de las entradas por clave

Las principales desventajas de los tries respecto a las tablas hash son:

- en determinados casos pueden ser más lentos que las tablas hash en la búsqueda de datos, especialmente si los datos son consultados desde dispositivos de almacenamiento secundario, como disco duro, donde el tiempo de acceso es elevado con respecto a memoria principal
- no es sencillo representar todas las claves como cadenas, como los números reales, que pueden tener distintas representaciones en forma de cadena para un mismo número, p.ej. 1, 1.00, 1.000, +1.000,...
- a menudo los tries son más ineficientes respecto al espacio que las tablas hash

- los tries no suelen estar disponibles con las herramientas de desarrollo software, todo lo contrario que las tablas hash

Compactación

Los tries (variantes) son estructuras muy adecuadas para la implementación de compactadores/compresores de datos.

FAMILIA DE TRIE

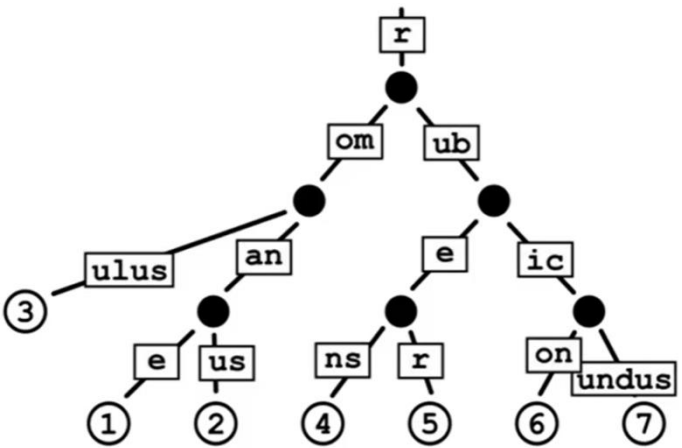
- Arboles Patricia
- Arboles Digitales
- Trie comprimido
- DAWG
- Two-Trie

Radix Tree (Arbol Patricia)

- Una optimización en espacio de un Trie.
- En vez de ir agregando una letra, poner todo lo que no necesite ramificacion en un nodo.

Ejemplo

1 romane
2 romanus
3 romulus
4 rubens
5 ruber
6 rubicon
7 rubicundus



Consultas…



Próximas clases

- Clases prácticas:
  - Viernes 4-11/11 – Miércoles 9/11
- Clase de teoría:
  - Tema: Árbol B
  - Viernes 16/11